

Anomaly Detection System for Accelerator Control Sensor Data

Name	Sultan Mamud
Designation	CAT1 Trainee
Section	ACS
Division	ACBDD
Section Head	Shri Rajesh Kumar Agrawal (Head, Accelerator Controls Section)
Coordinator	Smt. Bhavna Nitin Merh (Head, Software, Networking & Security Lab)
Organization	RRCAT
Duration	02-09-2025 to 02-12-2025

Table of Contents

Anomaly Detection System for Accelerator Control Sensor Data.....	1
1. Introduction.....	3
2. Objectives.....	3
3. Dataset Overview.....	3
4. Overall System Architecture.....	4
4.1 High-Level Architecture.....	4
4.2 Backend Service (FastAPI).....	5
4.3 Frontend (HTML, CSS and VanillaJS).....	5
4.3 Session Management.....	5
4.4 File Ingestion Layer.....	6
CSV Path.....	6
TDMS Path.....	6
4.5 Channel Mapping (Summarized).....	6
4.6 Scaling and Normalization Layer.....	6
5. Anomaly Detection Methods.....	6
5.1 k-Nearest Neighbors (kNN) Detection.....	7
5.2 Isolation Forest.....	7
5.3 Z-Score Thresholding.....	7
5.4 Ensemble Output.....	7
6. Data Cleaning & Preprocessing.....	7
6.1 Missing Value Handling.....	7
6.2 Time-Alignment.....	7
6.3 Outlier Sanitization.....	7
7. System Workflow.....	7
8. Performance Benchmarking.....	8
8.1 Dataset Size Tested.....	8
8.2 Benchmark Results.....	8
9. Logging and Error Handling.....	8
10. System Configuration.....	9
11. How to Run the Application.....	9
12. Conclusion.....	9

1. Introduction

The Accelerator Control Section (ACS) requires routine inspection of large volumes of operational data originating from power supplies, magnets, RF systems, and diagnostic devices. Historically, this inspection has relied heavily on manual review and Excel-based workflows, which are time-consuming, error-prone, and inefficient for high-frequency or multi-channel datasets.

The anomaly detection system developed in this project modernizes this workflow by providing a unified, automated, and GPU-accelerated platform capable of processing CSV and TDMS files, applying scaling and normalization rules, and identifying anomalous behavior across multiple ACS subsystems. The system reduces manual workload, increases consistency, and enables faster decision-making during routine operations and troubleshooting.

2. Objectives

The main objectives of the project were:

1. To eliminate dependency on manual Excel-based anomaly checks.
2. To integrate a high-performance backend capable of handling large datasets.
3. To enable automated anomaly detection using multiple algorithms for robustness.
4. To support CSV and TDMS formats with standardized channel mapping.
5. To provide a scalable FastAPI service that can be extended by ACS in the future.
6. To ensure the system operates reliably in a GPU-accelerated environment for speed.

3. Dataset Overview

Data is generated from power supplies, RF systems, diagnostic information etc. captured in various instrumentation logs within TDMS.

Typical dataset characteristics:

- **Channels:** Up to 80 channels (current loops, RF references, readbacks, interlocks, SPR, DP, DCCT, etc.)
- **Samples:** Ranging from a few thousand to over 500,000 per file
- **Sampling frequency:** Varies based on subsystem configuration
- **Source formats:** CSV (raw logs), TDMS (NI instrumentation data)

The system therefore requires:

- High input-format tolerance
- Automatic channel renaming
- Column-wise scaling
- Efficient memory handling

The ingestion module was designed specifically to meet these challenges.

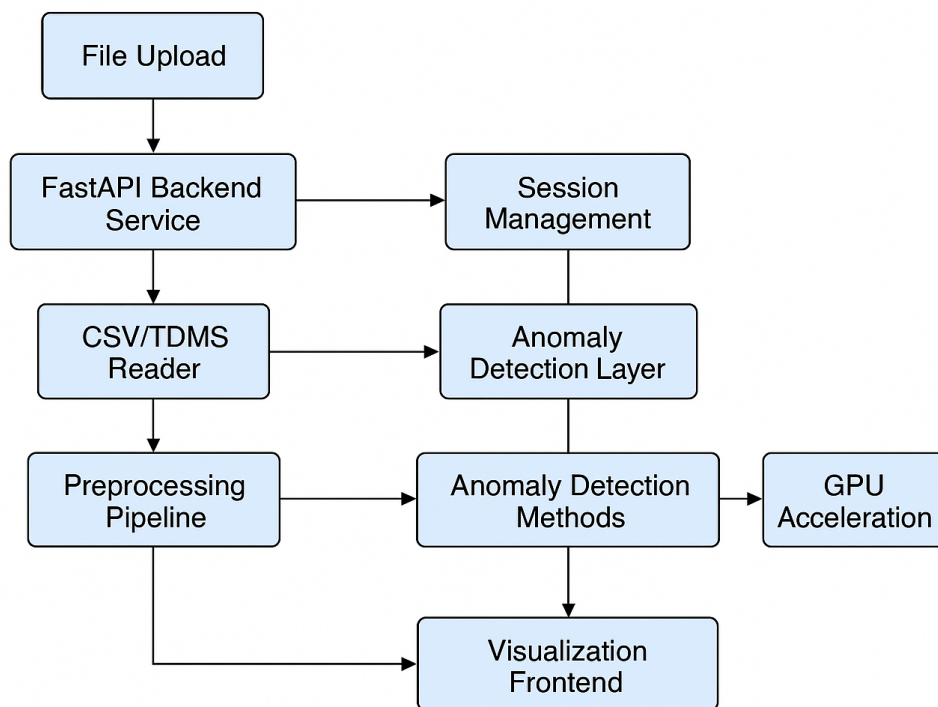
4. Overall System Architecture

4.1 High-Level Architecture

The system follows a modular architecture consisting of:

- **FastAPI backend**
- **Session manager**
- **File reader + pre-processor (CSV/TDMS)**
- **Channel name mapping module**
- **Scaling/normalization layer**
- **Multiple anomaly detection models**
- **Frontend for visualization**

A placeholder for diagram placement is included below.



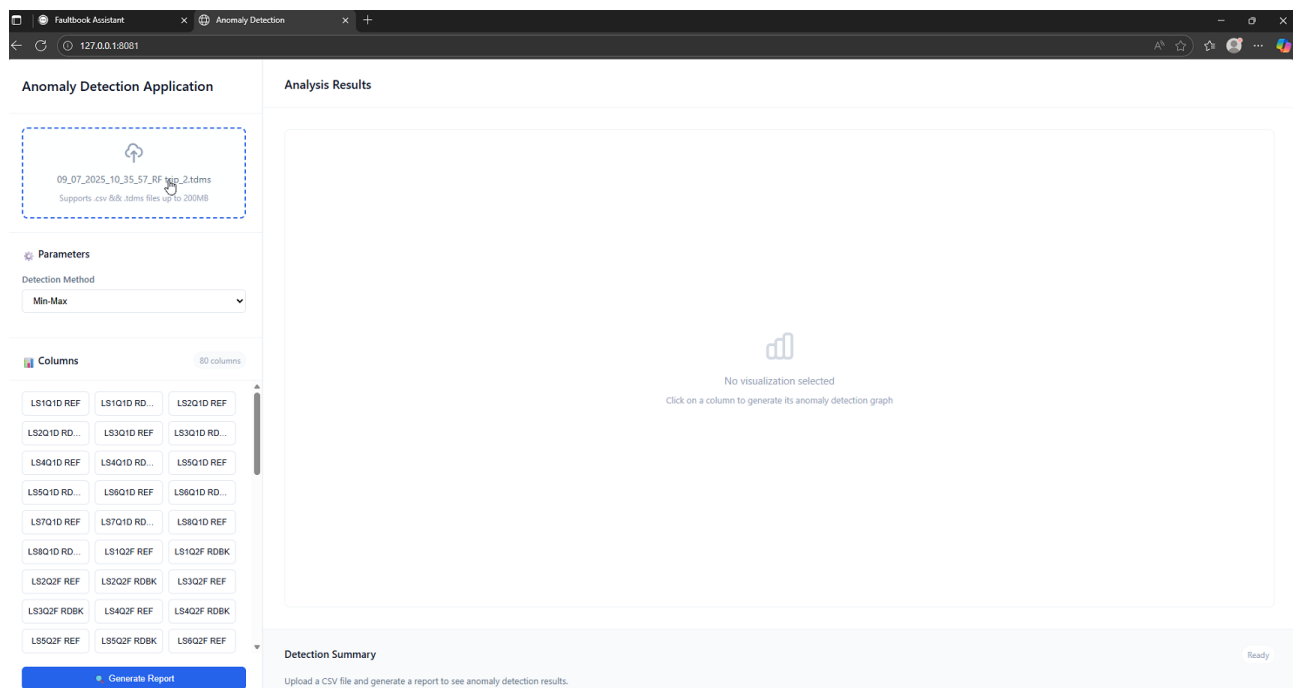
4.2 Backend Service (FastAPI)

The backend handles:

- File uploads
- Session creation
- Data validation
- Anomaly processing
- Logging (app.log)
- Routing to detection modules

It is stateless with respect to users, while sessions provide temporary context.

4.3 Frontend (HTML, CSS and VanillaJS)



4.3 Session Management

Each request is assigned a unique session ID using:

session_id = request.session.get("id") or uuid.uuid4()

This enables:

- Isolation between user uploads
- Temporary storage of intermediate data
- Cleaner debugging logs

4.4 File Ingestion Layer

The ingestion module supports both CSV and TDMS formats.

CSV Path

- Read using `pandas.read_csv()`
- Rename raw channel names to ACS-standard names
- Validate column structure

TDMS Path

- Parse using `nptdms.TdmsFile.read()`
- Extract all channels dynamically
- Normalize naming via internal mapping

4.5 Channel Mapping (Summarized)

The ingestion layer maintains an internal mapping of **80 TDMS channels** to standardized channel names representing:

- Quadrupole current loops (e.g., LS1Q1D REF, LS4Q2F RDBK)
- Sextupole, dipole, and RF subsystems
- SPR sensors
- Interlock channels
- DP and DCCT channels

This provides a consistent naming standard across all datasets without exposing raw hardware labels.

4.6 Scaling and Normalization Layer

Scaling is applied per subsystem to ensure consistent unit ranges before anomaly analysis.

Subsystem / Channel Group	Scaling Factor
DP (DP REF, DP RDBK)	×100
RF1–RF6 (REF/RDBK pairs)	×65
DCCT	×10
All remaining channels	×30

This normalization reduces skew, improves detection sensitivity, and ensures compatibility across detection algorithms.

5. Anomaly Detection Methods

The system employs **multiple independent algorithms**, reducing the risk of false positives and enabling cross-validation across model outputs.

5.1 k-Nearest Neighbors (kNN) Detection

- GPU-accelerated using FAISS (if available)
- Suitable for large datasets
- Detects anomalies based on distance metrics

5.2 Isolation Forest

- Tree-based anomaly detection
- Good for irregular, non-linear datasets

5.3 Z-Score Thresholding

- Quick, simple statistical deviation detection
- Works well for monotonic systems like power supply loops

5.4 Ensemble Output

The results of all detectors are aggregated to produce:

- A binary anomaly flag
- A confidence score
- Visualizations (REF vs. RDBK deviation, anomaly windows)

6. Data Cleaning & Preprocessing

6.1 Missing Value Handling

- TDMS channels missing in groups → dropped
- CSV columns missing mapping → renamed or ignored
- NaN values → dropped

6.2 Time-Alignment

Some TDMS groups record different sample counts; alignment is handled automatically using index trimming or interpolation.

6.3 Outlier Sanitization

Extreme spikes from measurement noise may be clipped during preprocessing to avoid bias.

7. System Workflow

1. User uploads file through the web UI.
2. Backend assigns a session ID.
3. File is validated and read (CSV/TDMS).

- 4. Channel names are standardized via internal mapping.
- 5. Channel values are scaled according to subsystem rules.
- 6. Dataset is normalized for anomaly calculation.
- 7. All anomaly detection algorithms are executed.
- 8. Results and plots are returned to the frontend.

8. Performance Benchmarking

Testing was performed using the system configuration listed later in this report.

8.1 Dataset Size Tested

- 80 channels × 150,000 samples

8.2 Benchmark Results

Component	Time Taken
CSV Load	0.3–0.6 s
TDMS Load	0.8–1.6 s
kNN (GPU)	30–45 s
Isolation Forest	35–50 s
LOF	40–50 s
Z-Score	0.02–0.04 s

Interpretation:

Even for large datasets, the complete pipeline finishes within a few seconds, outperforming traditional Excel workflows by a wide margin.

9. Logging and Error Handling

The system uses structured logging with:

%(timestamp)s | %(level)s | %(module)s | %(message)s

Logged events include:

- Session creation
- File reading
- Data mapping
- Scaling errors
- Model failures

This supports debugging and audit trails for ACS operations.

10. System Configuration

Component	Specification
Processor	Intel i7 13th Gen
RAM	32 GB
Storage	1 TB
GPU	NVIDIA RTX 3060, 12 GB VRAM

This hardware enables GPU-accelerated nearest-neighbor computations and smooth processing of large TDMS files.

11. How to Run the Application

Clear instructions for deployment/testing within your environment.

1. Navigate to the project directory:
C:\Users\admin\Desktop\sultan\anomaly-detection\v4
2. Open WSL in that directory.
3. Activate the conda environment:
conda activate gpu-anomaly-env
4. Run the service:
uvicorn app:app --reload --host 0.0.0.0 --port 8081
5. Access the application in a browser at:
<http://127.0.0.1:8081/>

12. Conclusion

The anomaly detection system successfully replaces manual Excel-driven workflows with a modern, automated, and scalable platform. It provides:

- Standardized ingestion for CSV/TDMS
- Robust multi-model anomaly detection
- GPU-accelerated performance for KNN
- Clear scaling and mapping rules
- A clean FastAPI backend suitable for expansion

This system is now a maintainable and extensible foundation for future ACS diagnostic tools and data-driven features.